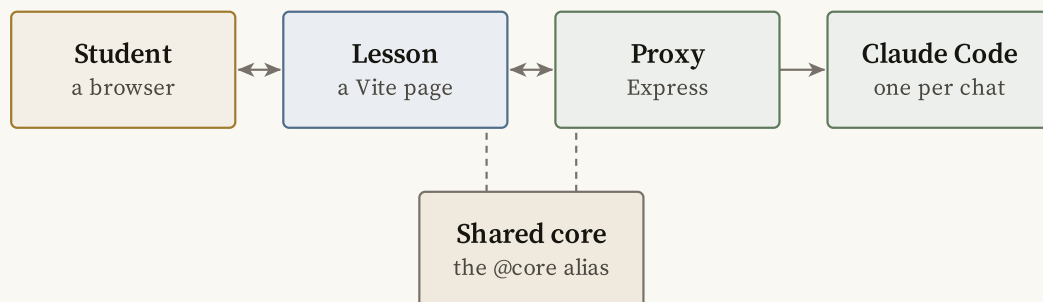


Lessons with a tutor inside

How lesson-builder turns course material into a lesson a student can talk to

lesson-builder is a Claude Code skill. Point it at a course's slides, notes and past problems and it builds a lesson: a small web app with the topics in order, the equations typeset, graphs the reader can move, animations, demos, and a tutor panel down the side that knows what the page says. This document is for someone who builds software and wants to know how that works. Every number in it was read off the skill's repo, or off the project where it is used, on 23 September 2026, and the grey line at the foot of each page says where.

FIGURE 1 *what runs when a student opens a lesson*



Ochre is the person, slate the page they read, sage the two processes behind it, and grey the code every lesson shares. Dashed lines are imports, not traffic.

A lesson is a Vite and React project, one per group of related topics, in a folder of its own under its course. It holds very little. The long file is the lesson's own text and figures; the page shell, the tutor chat, the maths renderer, the graph kit and the proxy all come from one shared core that every lesson imports. So a fix to the chat reaches every lesson at once, and a lesson stays readable as the thing it teaches.

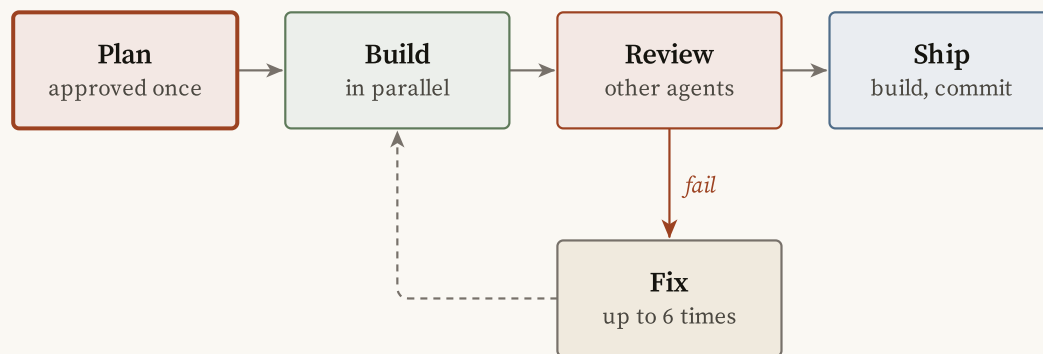
The layout and the shared core: SKILL.md and references/bootstrap/_lesson-core/. The proxy's routes: _lesson-core/server/proxy.js, lines 8 to 10.

Building one

A lesson is built in six phases by a team of agents, and the one approval a build cannot go without is a person's yes to the plan.

The first two phases read. An interview settles the scope: which materials count, whether the lesson may go beyond them, and where it ships. Then workers pull evidence out of the sources in parallel. The third phase writes the plan: what each topic should leave the student able to do, the order of moves that gets them there, and which medium carries each one, whether prose, a graph, an animation or a demo.

FIGURE 2 *from an approved plan to a shipped lesson*



Rust is a check that can say no, sage the work, slate what ships. The heavy box is the one place a person decides. The dashed line is work sent back.

Once the plan is approved, the rest runs on its own. Specialists build the media in parallel into a staging area; other agents review the result for code, content, correctness and how it looks on the page; and a fix loop runs until they pass or stop making progress, never more than six times. A reviewer's pass carries over to a later run only while nothing it read, and not its model, has changed. Only real, attributed problems from the course go in as practice. The skill will not invent one.

Every run keeps one record, a JSON file only one script touches, and the build log is rendered from it. An update builds in a git worktree of its own, so the author's checkout is never stashed or switched while a run is going.

The phases, the required gate and the six-round cap: SKILL.md, Pipeline and Execution guidance. The run record and reused verdicts: scripts/run-manifest.cjs, references/run-record.md and ROADMAP.md, item 10.

What a lesson is made of

Ten files of its own, and a shared core that does the rest.

TABLE 1 *the files a lesson keeps in its own folder*

FILE	WHAT IT DOES
src/<slug>.jsx	The lesson itself: its topics, its prose, and what the tutor is told about each topic
src/main.jsx	Five lines that mount it
index.html	The page it mounts into
package.json	Its own dependencies: React, Vite, KaTeX
server/proxy.js	One line that starts the shared proxy for this lesson
vite.config.js	Points the @core import at the shared core
test_lesson.cjs	Eighteen checks the lesson must pass before it ships
CLAUDE.md	What an agent editing this lesson needs to know
.gitignore	Keeps course material private by default
lesson_build.log.md	The build log, rendered from the run record

The shell is one component of 654 lines. It lays out the page with a contents rail that follows the reader down, a column for the text, and a tutor panel that docks at the side, floats over the page, or opens in a window of its own. Maths is set with KaTeX, loaded with an eight second limit, and if it never arrives the reader sees the LaTeX source rather than a blank.

The tutor can do more than answer. It talks back to the page through a handful of tagged messages: one changes a graph the student is looking at, one draws a demo, one proposes an addition to the lesson that the student can accept. A student can Ctrl-click any part of the page to put it into the chat, and can open a thread on a paragraph, which starts as a fork of the main conversation and is folded back into it as a summary when the student is done. Whatever the model writes is rendered through an allowlist of elements, never as raw HTML.

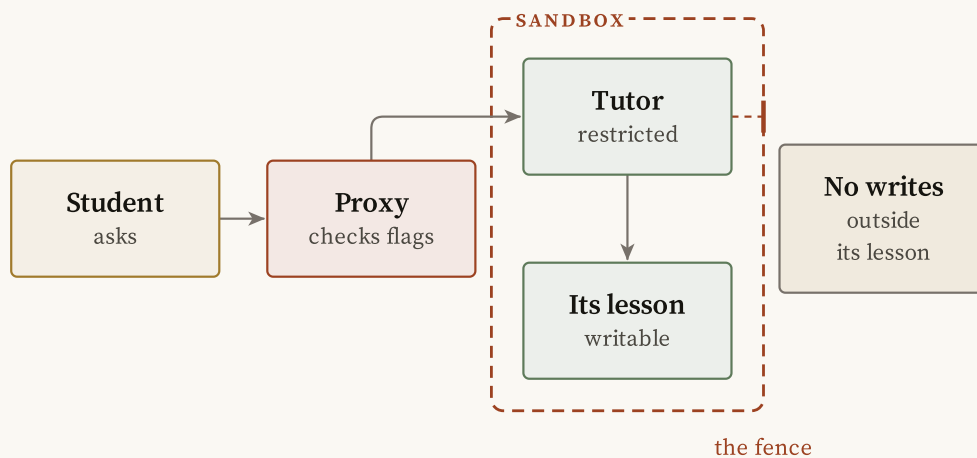
Table 1: SKILL.md, the per-lesson file list. The shell: _lesson-core/ui/LessonShell.jsx. The tutor's tags, threads and renderer: SKILL.md and _lesson-core/chat/safeRender.js.

A tutor that stays in its lesson

The tutor is a real Claude Code session, so what matters is what it cannot reach.

Each chat is its own Claude Code process, started by the lesson's proxy, so the tutor has real tools. Left alone, it would also see everything a developer's session sees.

FIGURE 3 *where the tutor may write*



Ochre is the student, rust the check that refuses, sage the tutor and the one folder it may write, and grey everything else. The rust dashed line is the fence.

On 16 September it did. Tutors in three courses loaded the operating notes of the machine they ran on, and one wrote an entry into a project journal outside its lesson. Now every tutor starts restricted: no CLAUDE.md is loaded, no hooks, no user MCP servers, writes to its own lesson only, and shell commands in a sandbox. If the CLI lacks those options, no tutor starts and every tutor route answers 503.

WHAT THE FENCE DOES NOT COVER YET

The fence bounds writes, not reads, so the tutor can still read files outside its lesson. And a file it writes can later be run by something that is not sandboxed. Both are known, and the fix waits on a decision.

The incident, 16 September 2026: `tests/tutor-confinement/README.md`. The flags and the refusal: `_lesson-core/server/proxy.js`, *Tutor confinement*. The open holes: the security review of pull request #47, 21 September.

How it is checked

Two layers of tests, and nothing that can block a merge needs a model or a browser.

18

checks every lesson
passes before it ships

21

test scenarios in the
skill's own gate

12

runs left out, each
printed with its
command

71

commits since 13 April

The first layer is inside each lesson. Its eighteen checks parse the file, confirm every topic has what the tutor needs, and build the tutor's whole prompt to see that it fits the CLI's argument list. Past 28,000 characters the proxy has to pass it on standard input instead, where it loses its priority as instructions, so that one is a test and not a guideline.

The second layer is the skill's own gate, a script every pull request must pass before it can merge. It runs 21 self-contained scenarios, mostly in parallel, with a fake `claude` where one needs the CLI. A scenario that leaves a server behind fails the gate. The runs that need a browser, the real CLI or a model are not skipped in silence: the gate prints each one with the exact command that runs it.

A person decides what a lesson teaches. Everything after that has something that can say no.

THE ONE CLAIM THIS DOCUMENT MAKES

**WHAT THE GATE DOES
NOT JUDGE**

Whether a lesson teaches well is judged by a reviewing model against a rubric. The evals that test that reviewer and the tutor on blind cases need a model too, so they are not in the gate, and a lesson can pass every test and still teach badly.

The eighteen checks: `references/checklists.md`. The gate, its fixtures and its exclusions: `tests/check.sh`. The ceiling: `_lesson-core/constants/promptBudget.js`. Teaching evals: `evals/teaching/README.md`. Commits: `git log`.

In use

It builds the lessons for a set of university courses, and most of what it has learned came from those lessons going wrong.

48	39	325
lessons, across 13 course folders	of them built before the current shell	commits to the lessons since 10 April

A built copy is published behind an access list, with the tutor live there since 7 September. A lesson can also have a printed companion, built by the skill that built this document.

TABLE 2 *what went wrong, and what changed*

WHAT HAPPENED	WHAT CHANGED
Cancelling a turn could kill the proxy itself	A process group is signalled only while the CLI still leads it
A dash in a long reply came out as a replacement character	The reader carries a character across a pipe-buffer boundary
A lesson built to the documented budget shipped a demoted prompt	The size is measured from the real prompt, and tested
A long reply was lost when the tab's stream dropped	The chat re-attaches to the turn still running
The tutor read the machine's operating notes	Every tutor starts restricted (page 4)

Each fix merged with a fixture in the gate that covers it. What is still open is shape: one app shell that mounts every lesson, typed actions in place of the tagged messages, and a tutor for the author kept apart from the student's.